

# Transport-Aware DNS Tunneling Detection

Classic DNS UDP/TCP and DNS-over-HTTPS Monitoring

CNS Project | Updated workflow report

**Abstract.** This project implements a transport-aware DNS tunneling detector with a terminal user interface for live and replay-based demonstrations. The system monitors classic DNS over UDP/53 and TCP/53, plus DNS-over-HTTPS traffic to known DoH resolvers on TCP/443. Packets are aggregated into short flows, converted into model features, routed to the correct detector, and displayed with a malicious probability, alert threshold, transport label, and high-signal feature summary.

## 1. Project Goal

DNS tunneling hides command-and-control traffic or data exfiltration inside DNS requests and responses. Classic tunnels often encode payloads into long subdomain labels, TXT records, NULL records, repeated query bursts, or unusual TCP DNS payloads. DoH tunnels are harder to inspect because DNS content is encrypted inside HTTPS, so detection relies on flow statistics such as packet sizes, timing, and request/response asymmetry.

- Support classic DNS over UDP/53 and TCP/53.
- Support conservative DoH monitoring for known resolver IPs.
- Use trained artifacts instead of simulated predictions.
- Provide a replay fallback so the demo works without root permissions.
- Expose confidence, model status, and high-signal features in the UI.

## 2. System Architecture

The final implementation centers on `pipeline/tui.py`. Live capture and replay mode share the same packet parser, flow extractor, model registry, thresholding rule, and UI path.

| Component         | Responsibility                                                                           |
|-------------------|------------------------------------------------------------------------------------------|
| Packet source     | Reads packets from live Scapy capture or from <code>samples/demo_dns_mix.pcap</code> .   |
| Flow aggregator   | Groups packets into short transport-aware flows, defaulting to 10-second windows.        |
| Feature extractor | Builds UDP53, TCP53, or DoH feature rows compatible with the deployed artifacts.         |
| Model registry    | Routes each flow to <code>artifacts/classic_dns/</code> or <code>artifacts/doh/</code> . |
| Decision layer    | Converts malicious probability into a UI label using a 0.70 threshold.                   |

|             |                                                                                   |
|-------------|-----------------------------------------------------------------------------------|
| Textual TUI | Displays alerts, model status, confidence, transport, and detailed flow features. |
|-------------|-----------------------------------------------------------------------------------|

### 3. Transport-Aware Routing

| Transport | Observed traffic                     | Artifact folder        | Model file     |
|-----------|--------------------------------------|------------------------|----------------|
| UDP53     | Classic DNS over UDP/53              | artifacts/classic_dns/ | best_model.pkl |
| TCP53     | Classic DNS over TCP/53              | artifacts/classic_dns/ | best_model.pkl |
| DOH       | TCP/443 flows to known DoH resolvers | artifacts/doh/         | xgboost.pkl    |

If a required artifact is unavailable, the UI reports `Unsupported model`. This prevents the demo from displaying a prediction that did not come from a trained model.

### 4. Feature Engineering

The classic DNS detector uses DNS-specific fields and derived ratios:

- Lexical query features: query length, entropy, label entropy, max label length, digit ratio, hex ratio.
- Record behavior: query type, response type, TXT/NULL flags, answer count, additional count.
- Timing and rate features: query rate and inter-arrival time.
- Payload relationships: payload-to-query ratio, response-to-query ratio, entropy per query character.
- TCP DNS fields: segment count, TCP payload size, DNS length field, retransmission pressure.

The DoH detector uses 28 statistical HTTPS flow features, including bytes sent and received, packet length distribution, packet timing distribution, and request/response timing asymmetry.

### 5. Model Training

Classic DNS training is implemented in `pipeline/train_classic_dns_detector.py`. It trains Logistic Regression, Random Forest, and XGBoost. The best classic DNS model is selected by F1 score, with recall and ROC-AUC as tie-breakers, and exported as `artifacts/classic_dns/best_model.pkl`. The DoH detector is trained and exported from `notebooks/doh_detector.ipynb`.

### 6. Current Model Results

#### Classic DNS

| Model         | Accuracy | Precision | Recall   | F1       | ROC-AUC  | Avg. Precision |
|---------------|----------|-----------|----------|----------|----------|----------------|
| Random Forest | 1.000000 | 1.000000  | 1.000000 | 1.000000 | 1.000000 | 1.000000       |

|                     |          |          |          |          |          |          |
|---------------------|----------|----------|----------|----------|----------|----------|
| XGBoost             | 0.999966 | 0.999976 | 0.999929 | 0.999953 | 1.000000 | 1.000000 |
| Logistic Regression | 0.999907 | 0.999953 | 0.999788 | 0.999870 | 1.000000 | 1.000000 |

At the TUI threshold of 0.70, the classic DNS artifact has precision 1.000000, recall 0.999976, F1 0.999988, and zero false positives on the exported threshold analysis split.

## DNS-over-HTTPS

| Model               | Accuracy | Precision | Recall | F1     | ROC-AUC | Avg. Precision |
|---------------------|----------|-----------|--------|--------|---------|----------------|
| Random Forest       | 0.9998   | 0.9999    | 0.9997 | 0.9998 | 1.0000  | 1.0000         |
| XGBoost             | 0.9999   | 1.0000    | 0.9999 | 0.9999 | 1.0000  | 1.0000         |
| Logistic Regression | 0.9014   | 0.8732    | 0.9423 | 0.9065 | 0.9583  | 0.9532         |

## 7. TUI Workflow

The TUI runs in three modes:

- **replay**: reads a local demo PCAP and runs the complete detection pipeline.
- **live**: sniffs live traffic from a selected network interface.
- **auto**: tries live capture first, then falls back to replay when live capture fails or no DNS/DoH packets are observed.

```
.venv/bin/python pipeline/tui.py --mode replay --pcap samples/demo_dns_mix.pcap
```

```
sudo .venv/bin/python pipeline/tui.py --mode live --iface <interface>
```

```
.venv/bin/python pipeline/tui.py --mode replay --malicious-threshold 0.70
```

## 8. GitHub Setup Workflow

```
git clone <repo-url>
cd c1-dns-tunneling
bash scripts/setup_env.sh
bash scripts/run_replay_demo.sh
```

Manual setup is also supported:

```
python3 -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip
python -m pip install -r requirements.txt
```

```
python pipeline/create_demo_pcap.py
python pipeline/tui.py --mode replay --pcap samples/demo_dns_mix.pcap
```

## 9. Demo Plan

---

1. Start replay mode.
2. Show that the table contains UDP53, TCP53, and DoH transports.
3. Select a benign row and point out normal query rate, record type, and lower malicious probability.
4. Select a malicious row and point out high entropy, long labels, TXT/NULL behavior, burst rate, or DoH flow statistics.
5. Explain that the same UI can run in live mode with packet-capture permissions.

## 10. Limitations And Future Work

---

The current system is designed for a class demo and prototype SOC monitor. Future improvements include adding more live TCP DNS captures to the classic DNS training data, testing on additional networks, adding persistent alert export, and integrating richer explanations such as feature contribution scores.

## 11. Conclusion

---

The final workflow provides a practical end-to-end DNS tunneling detector: trainable model artifacts, a transport-aware live/replay TUI, conservative model routing, thresholded alerts, and a reproducible GitHub setup path. Classic DNS and DoH are handled by different feature pipelines and artifact folders, while the UI presents the results through a single monitoring experience.